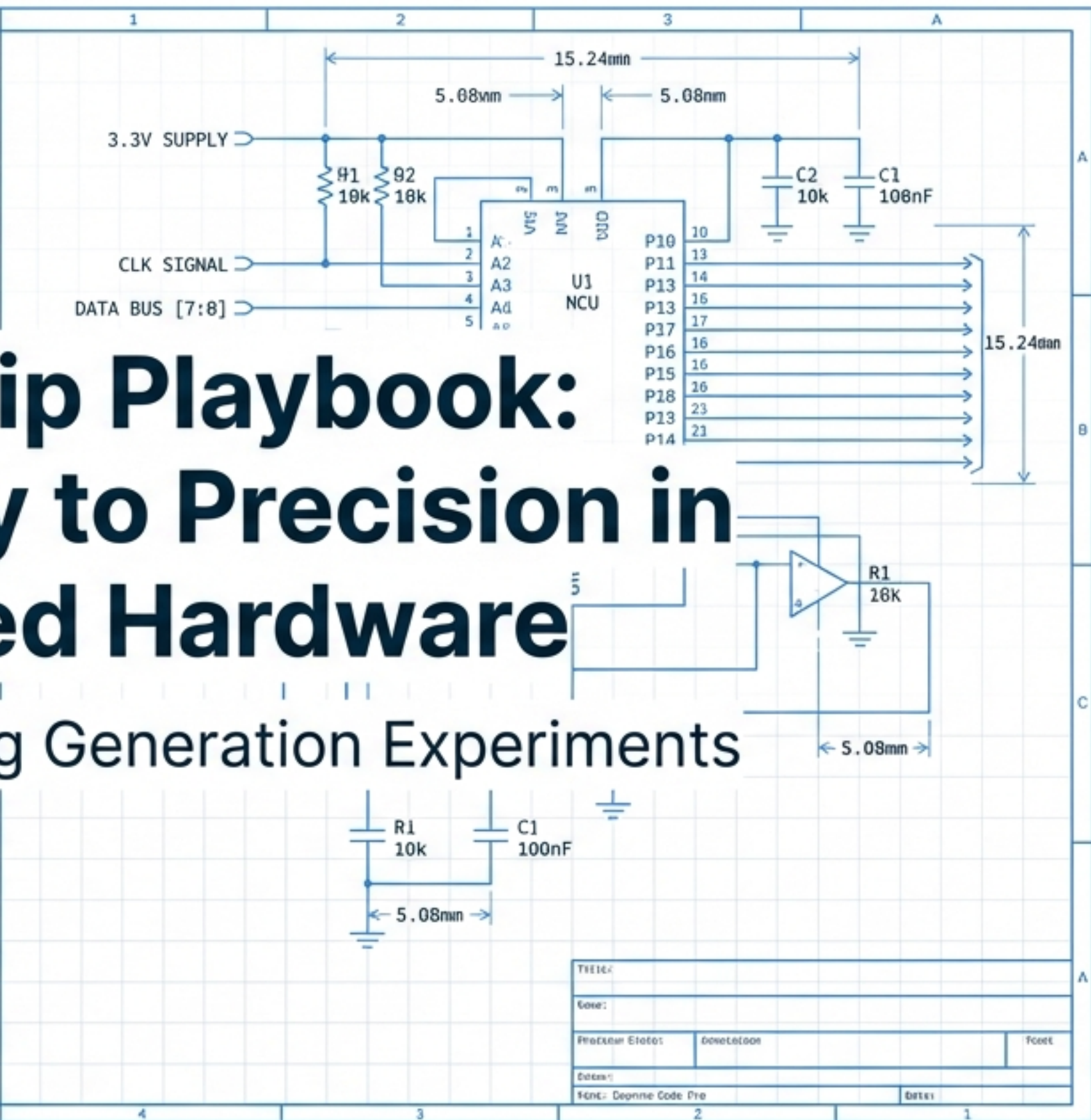
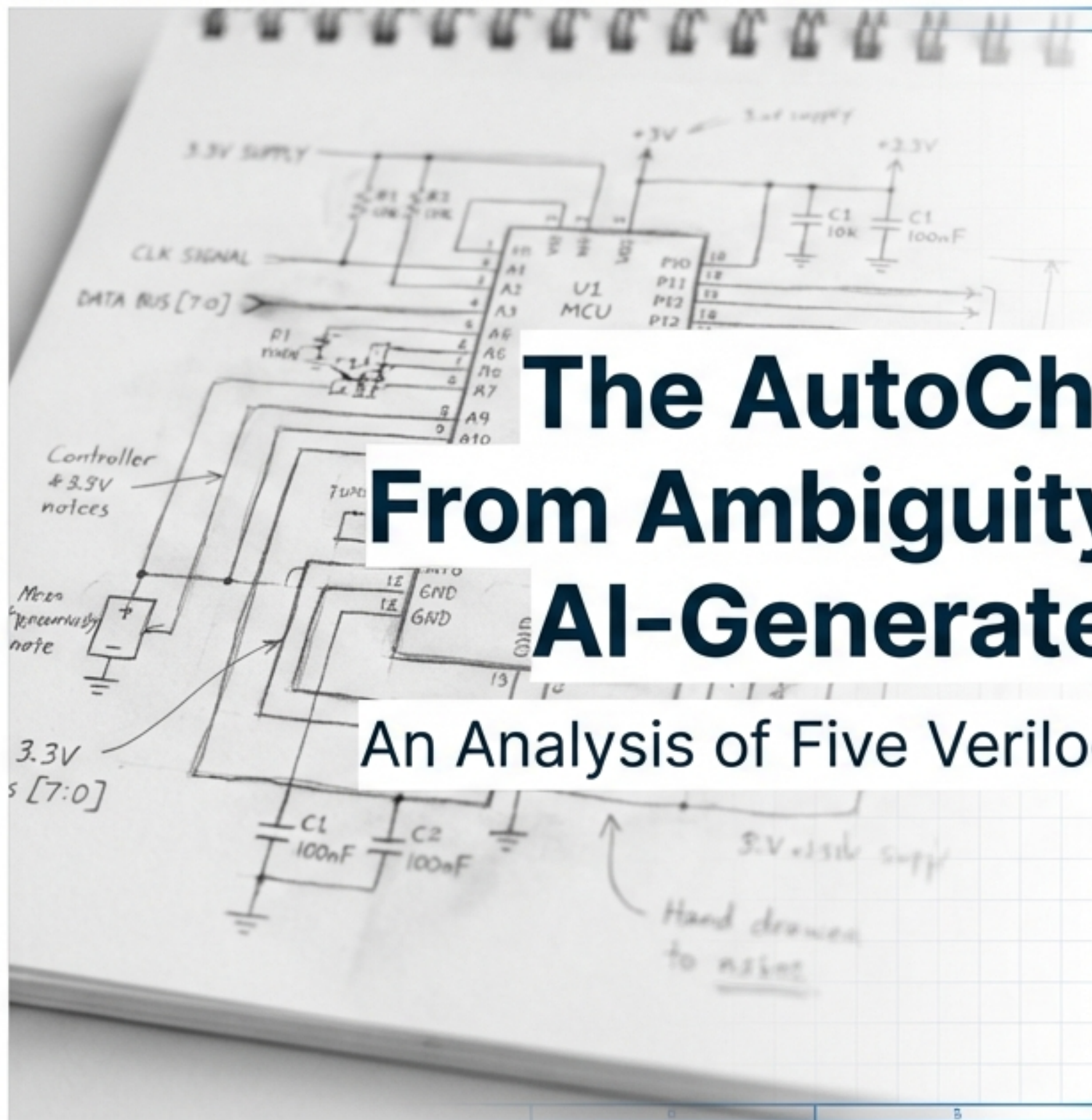




The AutoChip Playbook: From Ambiguity to Precision in AI-Generated Hardware

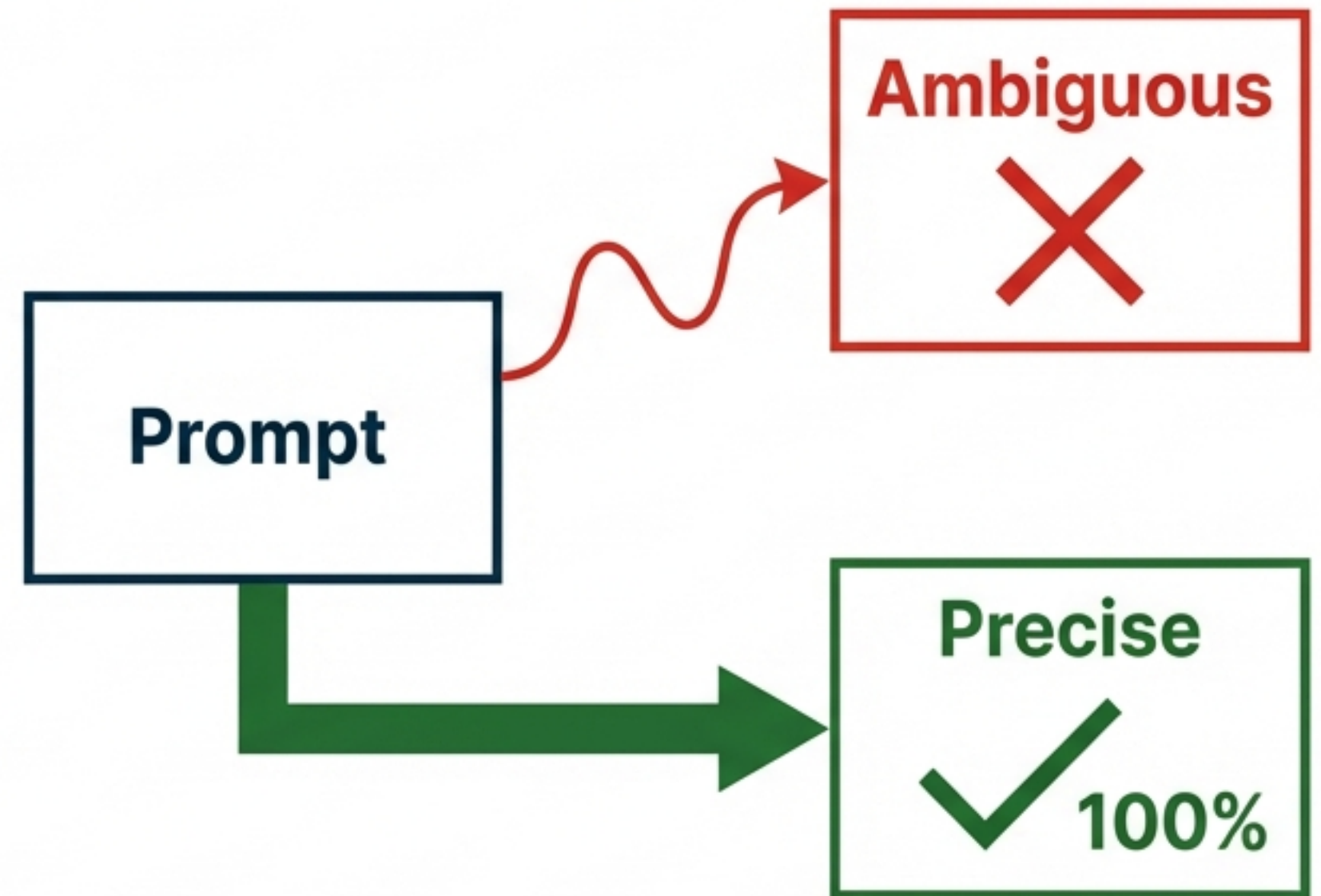
An Analysis of Five Verilog Generation Experiments

The Determinant of Success is Prompt Precision

Key Thesis Statement

Our experiments with the AutoChip framework reveal that an AI's ability to generate flawless Verilog is not a matter of chance. Success is almost entirely determined by the specificity and clarity of the prompt.

- **100% Success Rate:** Well-specified prompts with exact arithmetic, timing, and behavioral constraints consistently produced perfect results across all generated candidates.
- **Systemic Failure:** The single case of failure was not random. Every candidate exhibited the exact same error, tracing back to a single point of ambiguity in the prompt's timing requirements.
- **The Playbook:** By analyzing these successes and failures, we can distill a set of principles for crafting prompts that ensure reliable, high-quality hardware generation.



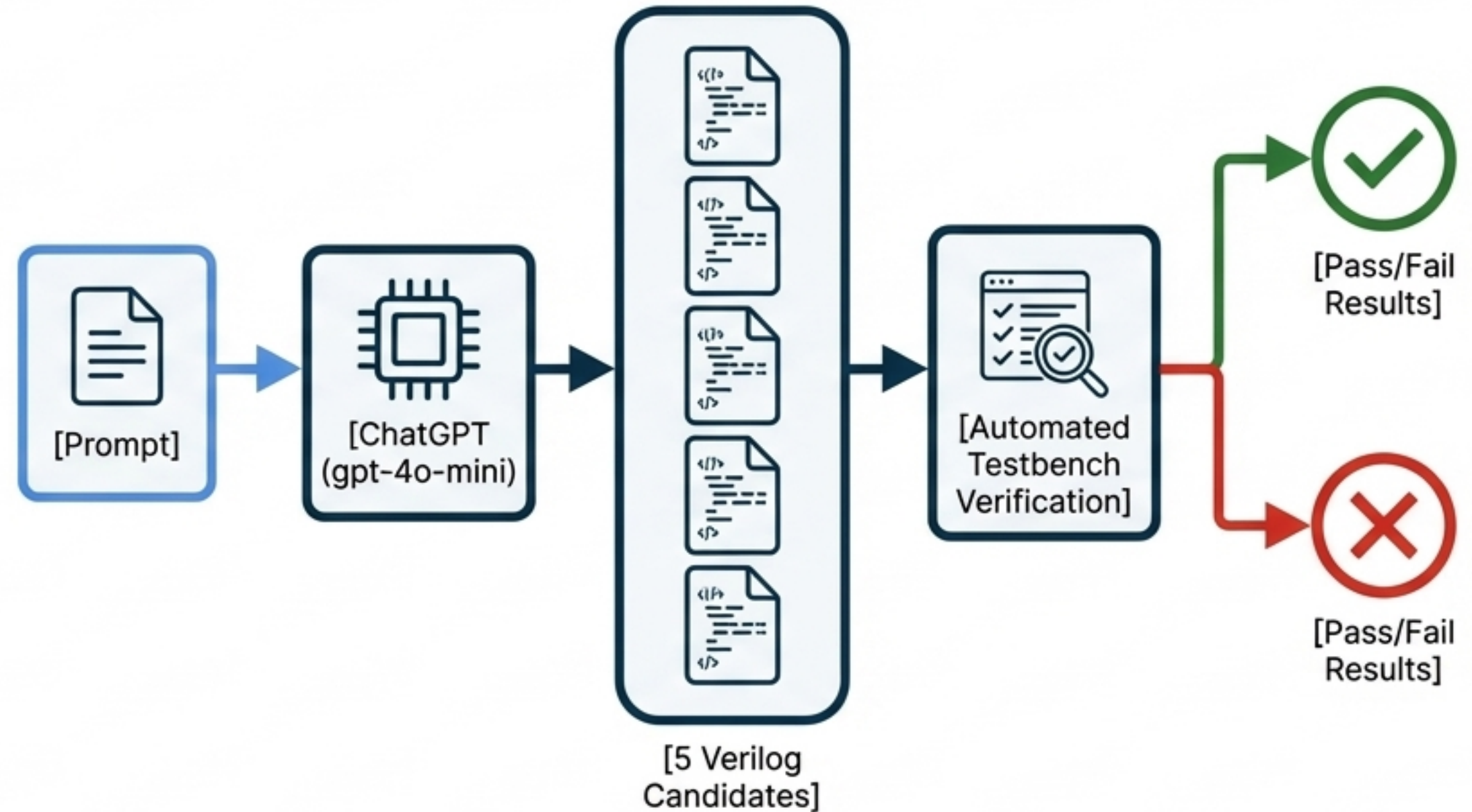
Our Experimental Framework: The AutoChip Loop

The Process

- **Model:** ChatGPT (gpt-4o-mini)
- **Method:** For each design, the AI generated 5 independent Verilog candidates based on a single prompt.
- **Verification:** Each candidate was automatically compiled and validated against a pre-written Verilog testbench.

The Metrics

- **Success:** Measured by Mismatches / Samples. A score of 0 / N indicates a perfect pass.
- **Consistency:** We assessed how many of the 5 candidates passed for each prompt.
- **Cost:** Approximate cost per candidate generation was tracked (e.g., \$0.0001 – \$0.0003).



The Power of Precision: When Prompts Are Unambiguous

We will first examine three distinct cases where highly specific prompts led to a 100% success rate across all AI-generated candidates, from simple arithmetic to complex sequential logic.

Case 1: Perfect Results from Simple, Exact Math

Module: Binary to BCD Converter

The Prompt's Core Instruction

Generate Verilog-2001 module...
converting 5-bit binary_input (0-31)
to 8-bit BCD... **using**
division/modulo.

Key Insight

Well-specified prompt with exact
header/ports + simple arithmetic led to
consistent, correct completions across
candidates.

Results Dashboard



Success Rate

100% (5 of 5 candidates passed)



Testbench Score

0 Mismatches / **32** Samples



Rank

1.0 (Perfect)

Case 2: Complex Logic, Flawless Execution

Module: Die Roller (LFSR-based RNG)

The Prompt's Core Instructions

```
Source Code Pro Regular
`...LFSR[7:0], roll rising-edge detection,
taps x^8+x^7+x^6+x^5+1, modulo mapping
for 4/6/8/20-sided dice`; single
`always` block, nonblocking ops.`
```

Key Insight

Precise behavioral spec (roll edge, modulo ranges 1..N, feedback taps) ensured deterministic compliance across candidates.

Results Dashboard



Success Rate: 100%
(5 of 5 candidates passed)



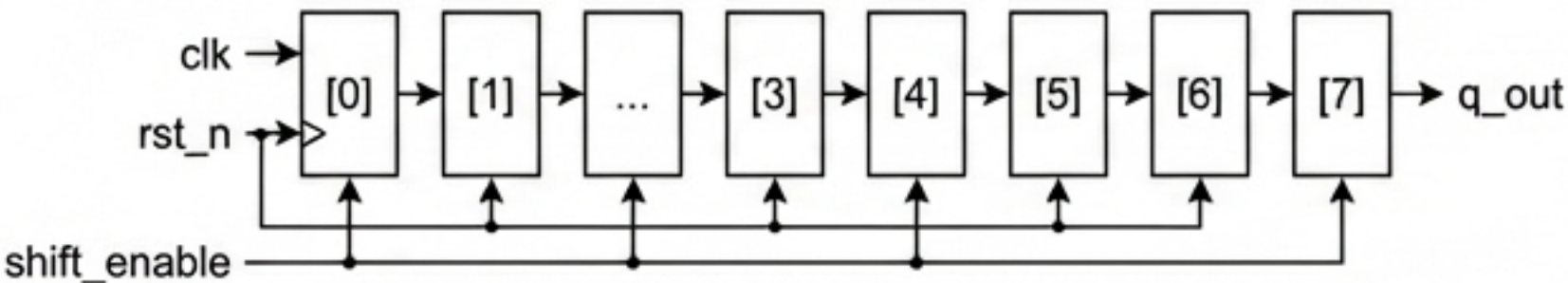
Testbench Score:
0 Mismatches / 4000 Samples



Rank: 1.0 (Perfect)

Case 3: Simplicity and Clarity Yield Robustness

Module: 8-bit Shift Register



The Prompt's Core Instructions

Generate a simple 8-bit shift register with **active-low reset** and **shift_enable**; output-only module...

Key Insight

Short, unambiguous prompt; simple sequential logic with reset/shift made it robust across completions.

Results Dashboard



Success Rate: 100%
(5 of 5 candidates passed)



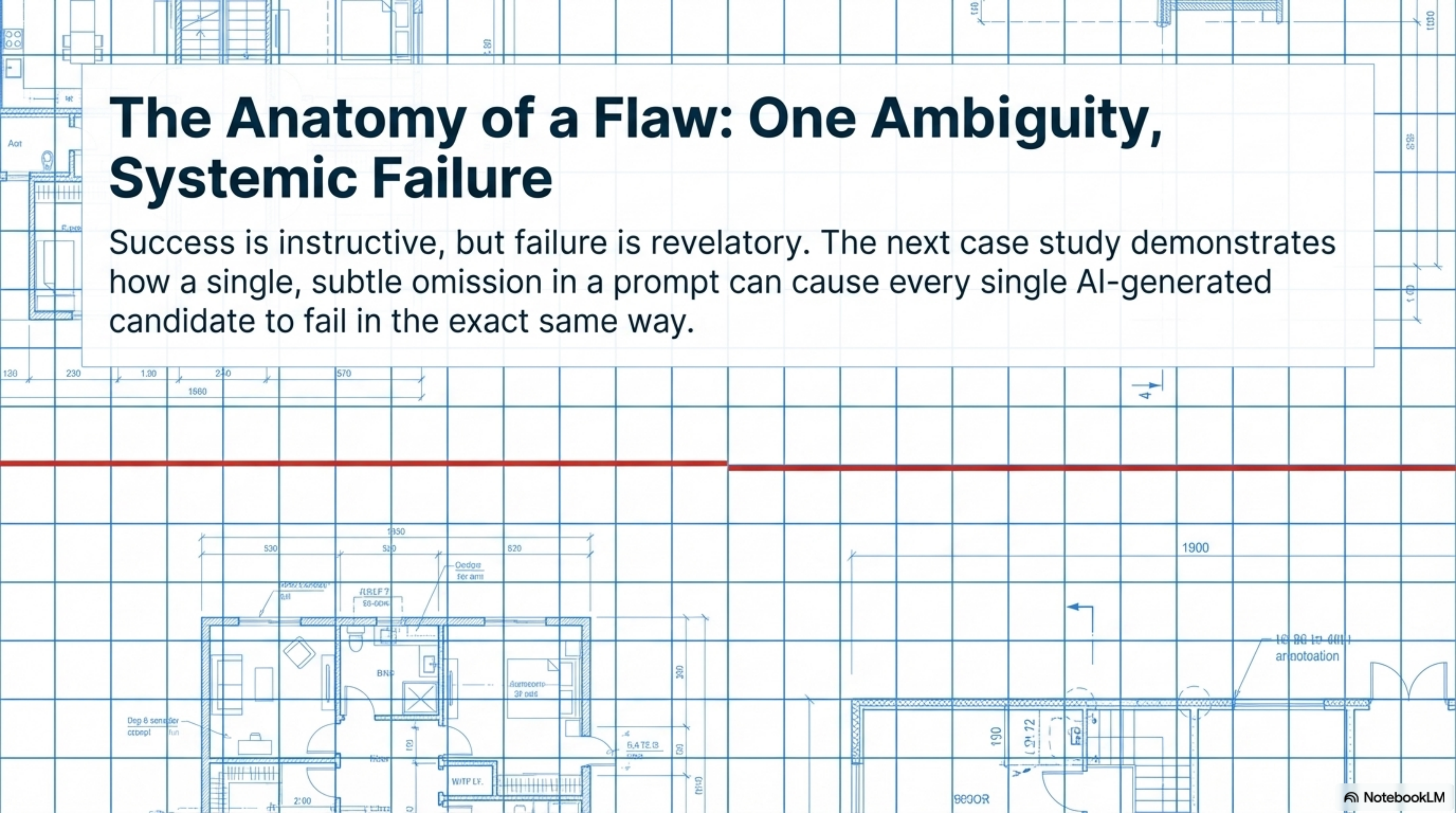
Testbench Score:
0 Mismatches / 7 Samples



Rank:
1.0 (Perfect)

[illegible]

Success is instructive, but failure is revelatory. The next case study demonstrates how a single, subtle omission in a prompt can cause every single AI-generated candidate to fail in the exact same way.



The Near Miss: A State Machine's Predictable Error

Module: State Sequence Detector (FSM)

The Prompt's Core Instructions

```
`Synthesize Verilog-2001 top_module  
with 8 states S0-S7 and exact  
transition rules; active-low async  
reset... combinational output  
`sequence_found=1` only when  
`state==S7` AND `data==3'b101`.
```

Key Insight

The identical error across all candidates points to a systemic flaw originating in the prompt, not random generation errors.

Results Dashboard



Success Rate: 0%
(0 of 5 candidates passed)



Testbench Score:
1 Mismatch / 12 Samples
(for all 5 candidates)

****Common Failure****

Error: Cycle 8, Expected: 1, Got: 0

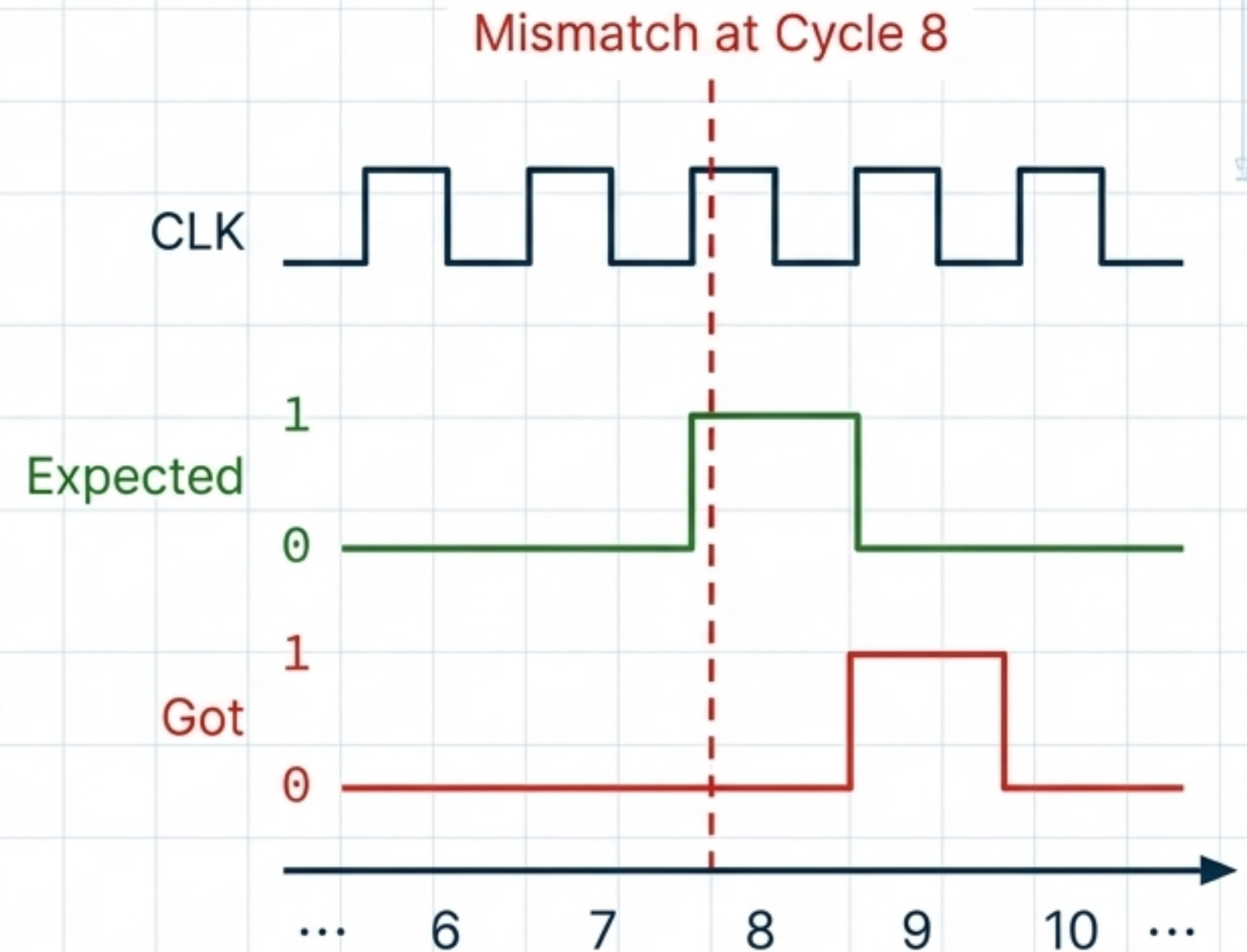
Pinpointing the Ambiguity: The Missing Timing Specification

The Flaw: The prompt specifies **when** the output should be true (``state==S7 AND data==3'b101``) but not **how** it should be timed relative to the clock cycle.

The Critical Omission: Is the output combinational (Mealy-style) or registered (Moore-style)? This ambiguity is the root cause of the failure.

The Consequence: All 5 candidates produced an output that was asserted one cycle too late or too early, resulting in the consistent **Error: Cycle 8, Expected: 1, Got: 0`**.

Insight from Report: "Tighten prompt to clarify Moore/Mealy timing and require explicit registered/comb structure for ``sequence_found``."



The AutoChip Playbook: Three Principles for Precision Prompting

From these experiments, a clear set of rules emerges. Following these three principles dramatically increases the probability of generating correct, robust hardware on the first attempt.

Principle 1: Mandate Specificity in Logic and Timing

The Rule:

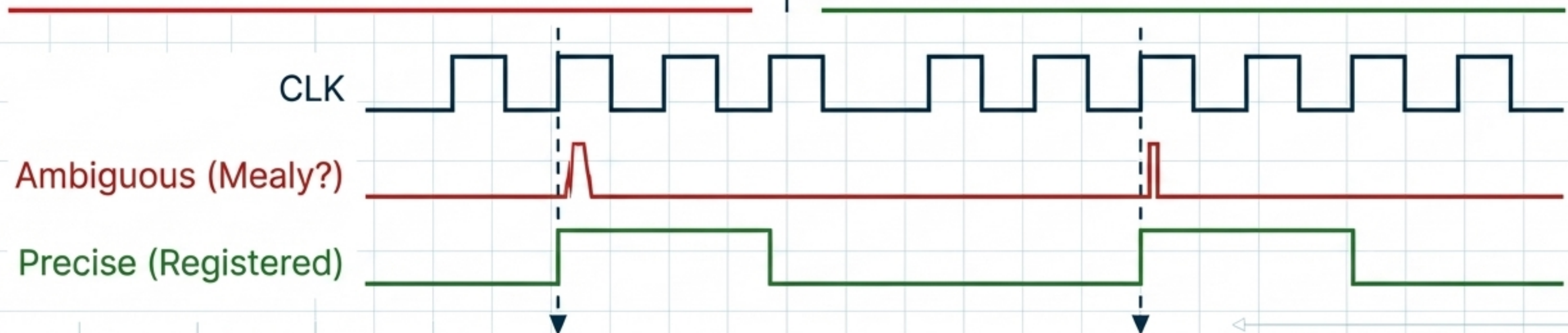
Never leave timing behavior implicit. For state machines, explicitly define Moore vs. Mealy output timing. For any sequential logic, specify if outputs should be combinational or registered.

Ambiguous Prompt

...combinational output ``sequence_found=1`` only when ``state==S7``...

Precise Prompt (Hypothetical)

...The output ``sequence_found`` must be a registered signal, asserted for one full clock cycle following the transition into S7 if ``data`` is ``3'b101``.



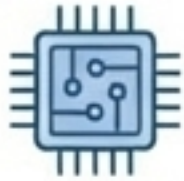
Principle 2: Define the Complete Operational Contract

The Rule:

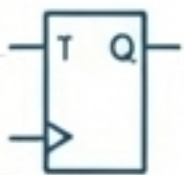
A prompt must act as a complete technical specification. Fix the module header, define reset behavior, and specify the exact arithmetic or algorithms to be used.



- **Binary-to-BCD:** Explicitly required ``division/modulo``, preventing other less robust implementations.



- **Die Roller:** Precisely defined the LFSR ``taps`` and ``modulo mapping`` rules, ensuring deterministic behavior.



- **Shift Register:** Clearly stated ``active-low reset`` and ``shift_enable``, leaving no room for interpretation.

“Strong, explicit prompts that fix module headers, reset behavior, and exact arithmetic/timing conditions lead to 100% pass rates.”

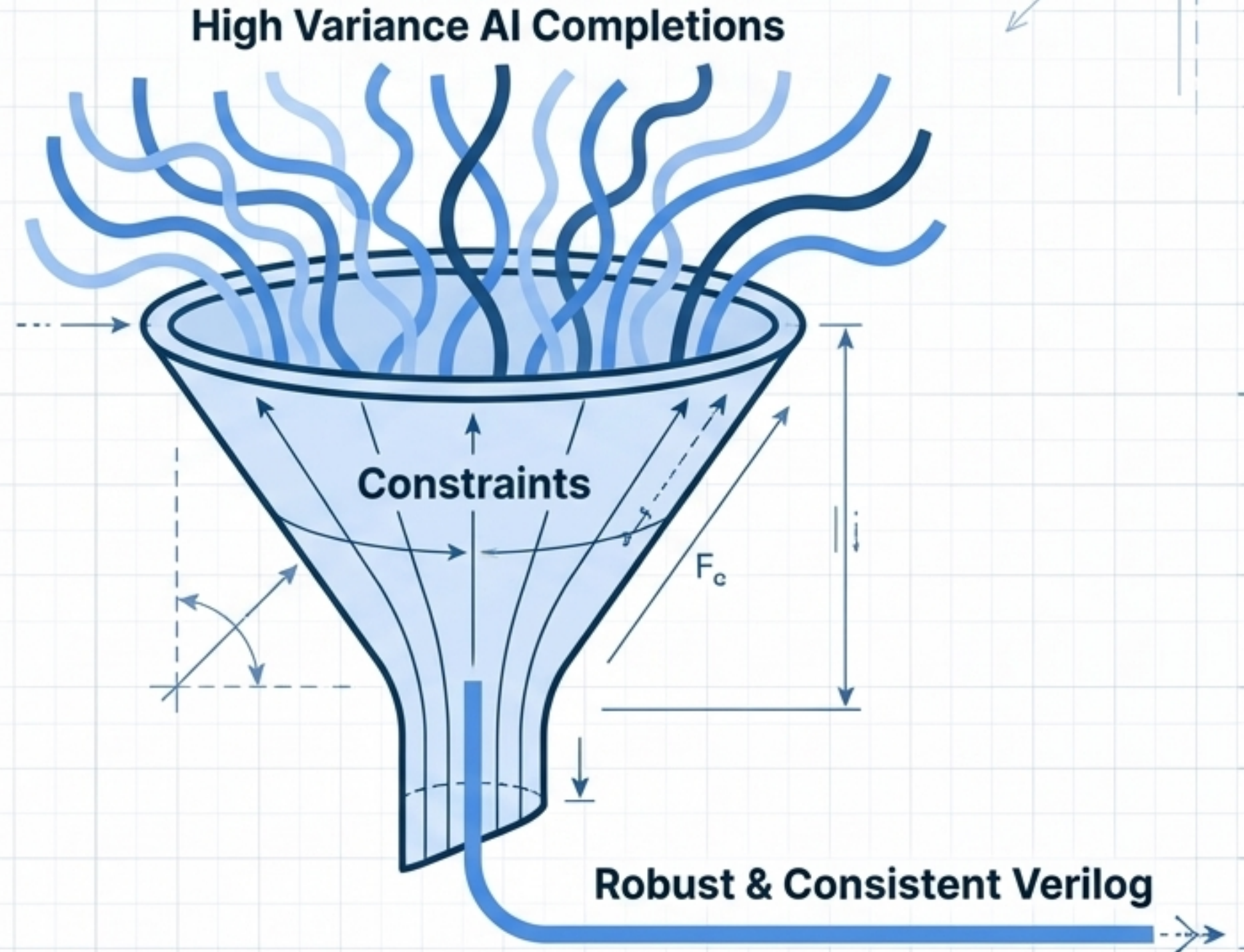
Principle 3: Constrain the Solution Space

The Rule:

Guide the AI's coding style and language feature usage to improve quality and consistency. These constraints reduce variance between candidates and prevent common simulation issues.

Actionable Constraints:

- Forbid SystemVerilog features (no SV).
- Enforce coding standards (nonblocking in sequential).
- Prevent non-synthesizable constructs (no initial blocks).



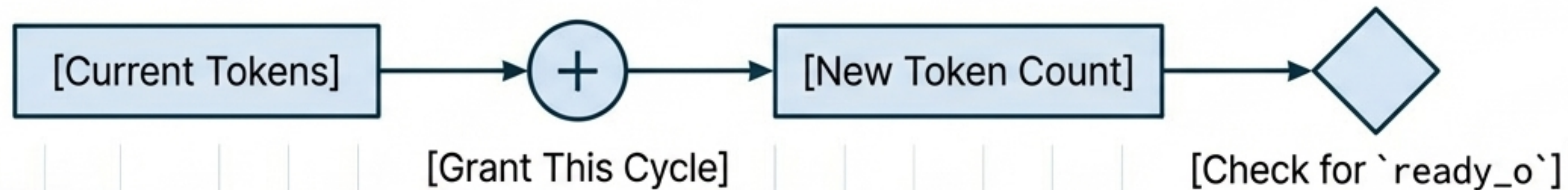
The Playbook in Action: The Token Bucket Rate Limiter

The Challenge: A more complex design with interacting parameters (DEN, `RATE\_NUM, `BURST_MAX) and potential for off-by-one errors.

Applying Principle 1 (Mandate Specificity):

The prompt contained a critical, precise instruction: “**POST-ADD semantics**”.

The Resulting Insight: This specific wording was key. It forced the AI to generate logic where the `ready_o signal is correctly based on the token count *after* the current transaction, preventing common grant/ready errors.



For complex hardware, identifying and specifying these critical semantic details in the prompt is the difference between a functional design and a subtly broken one. **Precision is paramount.**